

DOCUMENT PIPELINES & SEMANTIC SEARCH

Study Guide — From Extracted Text to Something Your AI Can Actually Search

Week 4 · Day 2 · Prepared by Rajesh Pasham

Program Context

Item	Detail
Session Type	Concepts and architecture lecture — no live Palantir platform work today
Platform	Individual Palantir Developer Tier accounts, used from Day 3 onward
This Week's Lab (Day 3)	Insurance Document Intelligence Pipeline — chunking, embedding, and searching real claim and policy documents

Where Today Fits

Day 1 got you a page of extracted text. That's not yet useful to an AI workflow at scale — a whole claim file is too long to hand an LLM every time, and exact keyword matching misses anything phrased differently. Today is about the two things that fix that: breaking text into meaningful pieces, and finding the right pieces by meaning, not just words.

Learning Objectives

By the end of today, you will be able to:

- Explain why documents get chunked before they're embedded, and what happens if a chunk is too large for the embedding model you chose
- Distinguish keyword search from vector search from hybrid retrieval, and know why combining methods usually beats picking just one
- Choose between giving an LLM the full document or just relevant chunks
- Explain how retrieval quality gets measured, and why citations matter

1. Why Chunk At All

- Embedding models have a maximum input length — many models accept only 512 tokens at a time, and a full claim file simply won't fit in one pass
- Smaller pieces are more semantically distinct during search — a chunk about “water damage to the kitchen” is easier to match precisely than a search across an entire 40-page claim file that also covers policy terms and payment history

2. The Chunk String Board

Chunking, concretely: one board in Pipeline Builder.

Setting	What It Means
What It Does	Breaks up a larger piece of text into smaller pieces — the mechanical step behind “chunking”
Expression	The text column you want to split — typically your Day 1 extracted-text output
Chunk Size	The approximate number of characters in each chunk — constrained by your embedding model's token limit

The result: each chunk gets its own associated Ontology object, linked back to the original document — the same Link Type discipline from Week 1, now applied to pieces of a document instead of whole records.

3. Choosing an Embedding Model

Not a purely technical choice — it shapes your chunking strategy:

- Token limits work backward into chunk size — most non-Ada embedding models accept only 512 tokens, so your Chunk Size setting has to respect whatever limit your chosen model has
- Symmetric vs. asymmetric matters for how you'll search — a model built for query-to-passage retrieval (like the models behind MS MARCO) fits a use case that starts from a short question; a model like Ada fits a use case that starts from a chunk and searches for similar chunks
- Generic models can underperform on domain-specific language — a general-purpose embedding model trained on broad web text may retrieve less precisely against dense insurance or medical terminology than expected, worth testing before committing

4. From Chunks to Searchable Ontology Objects

Each chunk becomes its own Ontology object, carrying a vector property that holds its embedding — this is what makes a chunk searchable by meaning, not just by exact text. The pipeline: extracted text → the Chunk String board → chunks sized to fit the embedding model → each chunk embedded → each chunk becomes its own Ontology object.

5. Vector Search: Nearest Neighbors in Embedding Space

- A query gets embedded the same way a chunk does, then the search finds the chunks whose vectors sit closest to the query's vector — “nearest neighbors” in that embedding space
- HyDE (Hypothetical Document Embeddings) bridges the gap when a query and a chunk don't naturally look alike — generate a plausible answer with an LLM first, embed that instead of the raw query, then search; useful because a chunk and an answer are more alike than a chunk and a question

6. Hybrid Retrieval: Combining Multiple Methods

No single method wins every time. Four real retrieval methods: keyword search, vector cosine (nearest-neighbor search in embedding space), augmented keyword search (an LLM generates likely keywords and synonyms first), and HyDE. Keyword search catches exact terms vector search might miss; vector search catches meaning keyword search can't see. Combining results from more than one method, via rank fusion, is usually more reliable than trusting any single approach.

7. Full Document vs. Relevant Chunks

Mode	What It Means
Full Document Text Mode	Gives the LLM the entire document as context — complete, but expensive in tokens, and impractical for a long claim file
Relevant Chunks Mode	Performs a semantic search over the document and returns only the K most relevant chunks — efficient, but only as good as the retrieval that selected those chunks

The practical rule: short, single documents can often use full-document mode. Anything long, or any workflow searching across many documents, needs chunk-based retrieval to stay both accurate and affordable.

8. Retrieval Quality Evaluation

The same evaluation discipline from Week 2, now aimed at retrieval specifically. An evaluation suite can compare retrieval techniques directly — benchmarking keyword search, vector cosine, augmented keyword search, and HyDE against the same set of test questions, to see which retrieves the most relevant chunks. A wrong answer is often a retrieval failure, not a reasoning failure: if a tool fails to answer something that should be in the document corpus, check first whether the right context was ever retrieved — the LLM can only reason over what it was actually given.

9. Citation & Provenance Tracking

Every retrieved chunk should trace back to its source document — the same citation discipline from Week 2's groundedReferences field, now anchored to a specific chunk of a specific file rather than an Ontology object. Surfacing the original document alongside the answer builds trust: rendering the source PDF next to a retrieved chunk lets a claims handler or underwriter cross-check the answer against the actual document, not just take the AI's word for it.

10. Common Pitfalls

- Chunking without checking the embedding model's token limit — a chunk sized larger than the model accepts gets silently truncated or rejected; check the limit first, then set chunk size
- Assuming vector search alone is always better than keyword search — vector search can miss an exact term match that a simple keyword search would have caught; hybrid retrieval exists because neither method wins every time
- Defaulting to full-document context out of convenience — it works for a short policy summary and fails, expensively, for a 40-page claim file; match the mode to the document length
- Treating retrieval as untestable — a retrieval-specific evaluation suite, benchmarking multiple methods against real questions, is exactly as buildable as the workflow evaluation suites from Week 2

11. Glossary

Term	Meaning
Chunking	Breaking a larger piece of text into smaller pieces, sized to fit an embedding model's input limit
Embedding	A numeric vector representation of text, positioned so similar meanings sit close together in vector space
Vector Search	Finding the nearest-neighbor chunks to a query's embedding in vector space

Term	Meaning
HyDE	Hypothetical Document Embeddings — embedding an LLM-generated hypothetical answer instead of the raw query, to improve retrieval
Hybrid Retrieval	Combining multiple retrieval methods (such as keyword and vector search) and merging their results via rank fusion
Rank Fusion	The technique for combining results from multiple retrieval methods into one ranked list
Full Document Text Mode	Giving an LLM an entire document as context
Relevant Chunks Mode	Giving an LLM only the top-K chunks returned by a semantic search

12. Further Practice

Continue into Day 3, the self-paced lab where you build the Insurance Document Intelligence Pipeline — applying today's chunking, embedding, and hybrid retrieval concepts to real claim and policy documents, linked back to your Week 1 Ontology.